

CIS 5357

Computing for Data Analytics

Week 3

1 FEBRUARY 2024

Last week

- Python comments, printing, variables, data types & math
- Debug common errors
- Take input from a user

Quick review

- What function allows you to capture user responses? `input()`
- How do you comment out lines of code? `line begins with #`
- How do you print on multiple lines without having to use the `print()` function multiple times? `\n`
- Is `9times` a valid variable name in Python? `no`
- How can you format a variable within a string? `f-string`
- What will `print(5 > 4)` return? `True`

This week

- selection and decision statements in Python
- if else logic
- exception handling

Download code to follow along

Module 3

Read and View

Read

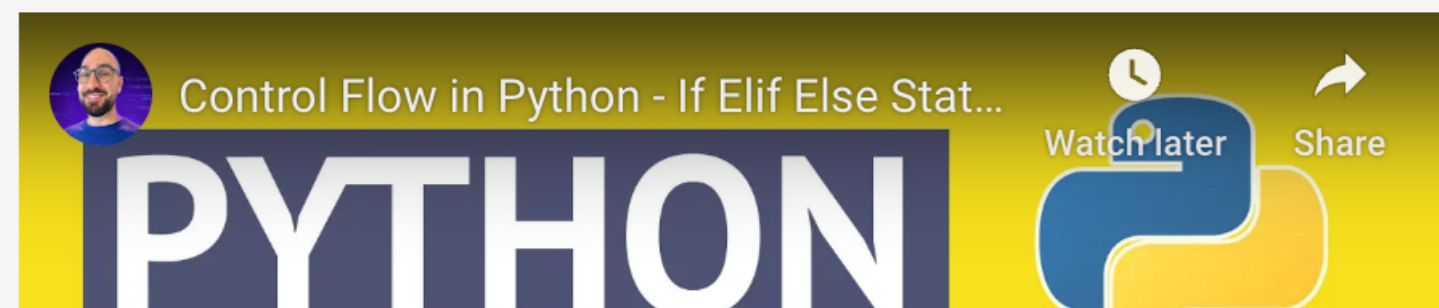
- Textbook chapter 3 pages 71-87
- [Rubber Duck Problem Solving](#) ↗
- [Python if statements \(section 4.1\)](#) ↗
- [Errors and exceptions in Python](#) ↗

For reference:

- [Practice code for lecture](#) ↓ —follow along in class
- Lecture slides (coming soon)

View

[If Elif Else Statements](#) ↗



Equality in Python

variable assignment is done with a single equals operator

```
age = 20
```

whereas comparison between two variables is done using the double equals operator

```
age == 20
```

```
# this is a logic statement that returns True if age  
# is equal to 20 and False otherwise
```


Inequality in Python

the “not equals” operator is marked as “!=”

```
age != 20  
# this will return False when age is 20 and  
# True otherwise
```

the *not* operator takes the True/False value and flips it

```
not age == 20  
# this will also return False when age is 20 and  
# True otherwise
```

Boolean values

The boolean values *True* and *False* are returned when an expression is compared or evaluated. Note *True* and *False* are capitalized—lowercase instances would be treated as variables in Python

```
12     print(age == 20) # prints out True
13     print(age == 30) # prints out False
14     print(age < 30) # prints out True
15
```

* Assuming that we'd already ran the code `age = 20`

Other comparison operators

We can check greater than (>), less than (<), and also greater than or equal to (>=), and less than or equal to (<=) something else

```
16     print(age < 20) # prints out False
17     print(age <= 20) # prints out True
18     print(age > 20) # prints out False
19     print(age >= 20) # prints out True
```

* Assuming that we'd already ran the code `age = 20`

Combining logic checks

and only returns True if every statement is True

```
21 username = "admin"
22 password = "12345"
23 username == "admin" and password == "12345" #what does this return?
```

or returns True if any statement is True

```
25 answer = "yes"
26 answer == "yes" or answer == "Yes" or answer == "YES" or answer == "Y" #what does this return?
--
```

The *in* operator

in checks if one variable is within another

```
< /
28     "d" in "hello"
29     "d" in "goodbye"
30     "d" in "GOODBYE"
> /
```

*note that *in* is case sensitive like other string operators

Quick logic practice

What Boolean value will each of these statements return (enter your response in chat):

```
>>> true and true
```

```
>>> True and True
```

```
>>> (1==1) and (2==1)
```

```
>>> not (10 == 100/10)
```

```
>>> 3==3 and not "Python" == "Fun"
```

```
>>> 1 !=10 or 3==4
```

Conditional statements

Code that runs only if the all of the defined conditions are met

```
42  answer = input("Do you want to hear a joke?")
43  if answer == "yes" or answer == "Yes" or answer == "YES" or answer == "Y":
44      print("what's loud and sounds like an apple?")
45      print("AN APPLE")
```

* Python uses indentation to define code blocks, instead of brackets. The **standard indentation is 4 spaces, although tabs and any other space size will work**, as long as it is **consistent**. Notice that code blocks do not need any termination.

If, else and elif statement

```
statement = False
another_statement = True
if statement is True:
    # do something
    pass
elif another_statement is True: # else if
    # do something else
    pass
else:
    # do another thing
    pass
```


If, else and elif statement

```
statement = False
another_statement = True
if statement is True:
    # do something
    pass
elif another_statement is True: # else if
    # do something else
    pass
else:
    # do another thing
    pass
```

Note the required colons

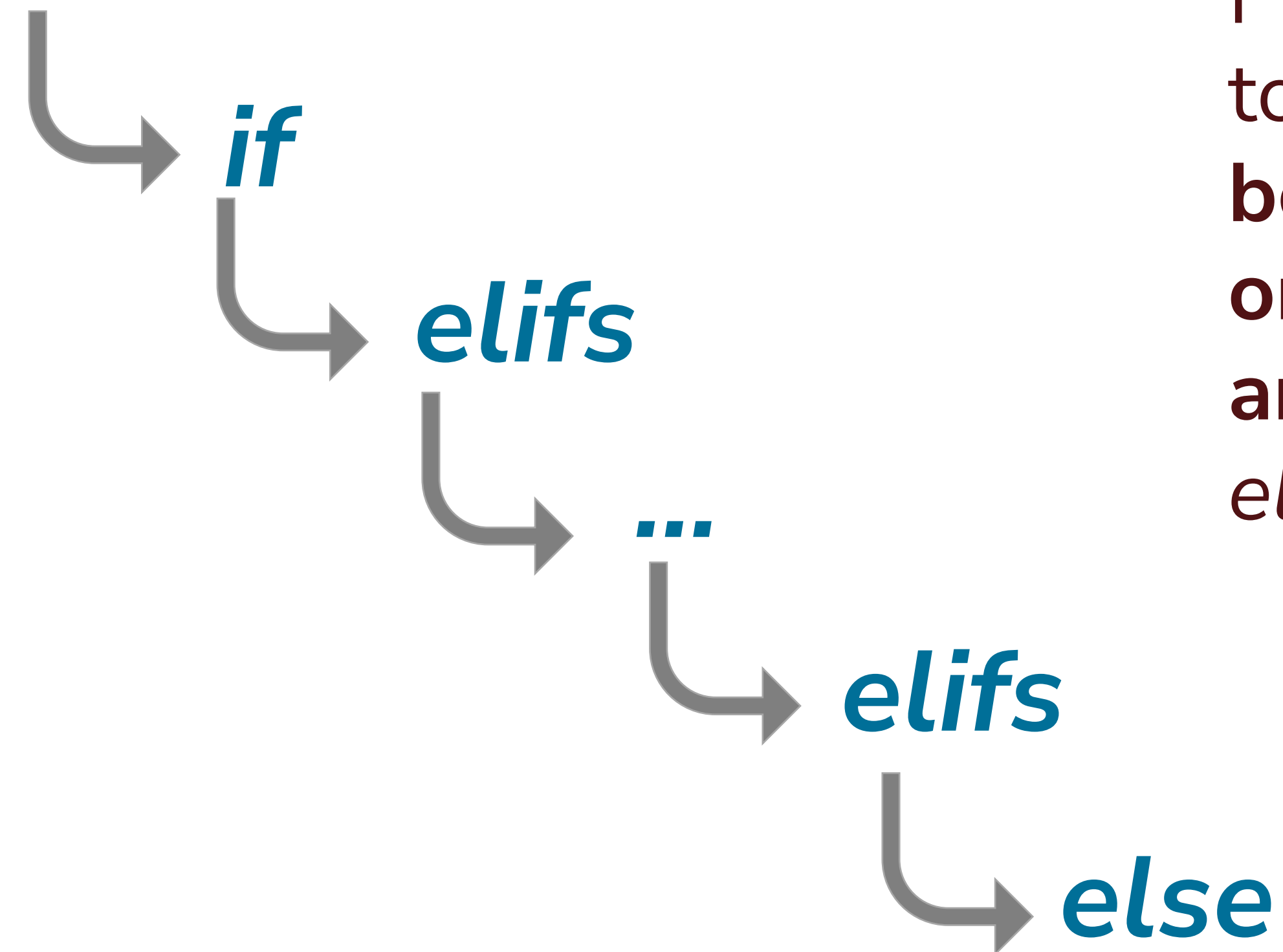
& indents

If, else and elif statement

```
statement = False
another_statement = True
if statement is True:
    # do something
    pass
elif another_statement is True: # else if
    # do something else
    pass
else:
    # do another thing
    pass
```

We can put as many *elifs* as we want in the middle but we can only have one *if* and *else*

Waterfall process



Python will start with *if* and moves down to the **next condition only when the one before it was not true**. Once it finds an *if* or *elif* that is true, Python will not check any of the ones after it. The code under *else* only runs as a last case scenario.

Ternary operator

useful in cases where we need to assign a value to a variable based on a simple condition, and we want to keep our code more concise — all in just one line of code.

```
value_if_true if condition else value_if_false
```

Ternary operator

useful in cases where we need to assign a value to a variable based on a simple condition, and we want to keep our code more concise — all in just one line of code.

```
value_if_true if condition else value_if_false
```

```
a, b = 10, 20
```

```
# Copy value of a in min if a < b else copy b
```

```
min = a if a < b else b
```

```
print(min)
```


Ternary operator

useful in cases where we need to assign a value to a variable based on a simple condition, and we want to keep our code more concise — all in just one line of code.

```
value_if_true if condition else value_if_false
```

```
a, b = 10, 20
```

← A quick way to assign multiple variables

```
# Copy value of a in min if a < b else copy b
```

```
min = a if a < b else b
```

```
print(min)
```

What will this return? *answer in chat*

useful in cases where we need to assign a value to a variable based on a simple condition, and we want to keep our code more concise — all in just one line of code.

```
value_if_true if condition else value_if_false
```

```
a, b = 10, 20
```

```
# Copy value of a in min if a < b else copy b
```

```
min = a if a < b else b
```

```
print(min)
```

Nested ternary operator

```
# Python program to demonstrate nested ternary operator
```

```
a, b = 10, 20
```

```
print("Both a and b are equal" if a == b else "a is greater  
than b" if a > b else "b is greater than a")
```

* Note how the logic can be used directly within a *print()* statement

Remember from last week

What if the user input was unexpected?

How much is the item?

- *apple*
- *(user hits enter/return key)*
- *\$6.50*

Python exceptions

Errors that occur at runtime (after passing the syntax test) are called **exceptions** or **logical errors**.

Python exceptions

Errors that occur at runtime (after passing the syntax test) are called **exceptions** or **logical errors**.

For example

- try to open a file(for reading) that does not exist (*FileNotFoundError*)
- try to divide a number by zero (*ZeroDivisionError*)
- try to import a module that does not exist (*ImportError*)

*reference documentation for a comprehensive list of exceptions

Python exceptions handling

Exceptions can be caught and handled by the program using **try except** blocks

```
66  try:
67      # code that may cause exception
68  except:
69      # code to run when exception occurs
70  else:
71      # code to run when no exception occurs
72  & indents
```

Note the required colons

For each *try* block, there can be zero or more *except* blocks.
Multiple *except* blocks allow you to handle each exception differently.

Discount challenge example

Write Python code that asks a user for the price of an item, then calculate these discount amounts: **10%, 15%, 20% and 25%** and the **final sale price for each discount** amount.

```
try:
    amount = float(input("How much is the item? "))
except ValueError:
    print("Sorry, that is not a valid number")
else:
    # then calculate the discount amounts
    print(f"10% off is ${amount * .10:.2f}; the sale price is ${amount-(amount * .10):.2f}")
    print(f"15% off is ${amount * .15:.2f}; the sale price is ${amount-(amount * .15):.2f}")
    print(f"20% off is ${amount * .20:.2f}; the sale price is ${amount-(amount * .20):.2f}")
    print(f"25% off is ${amount * .25:.2f}; the sale price is ${amount-(amount * .25):.2f}")
```

Challenge: temp check

Ask a user the temperature of water in Celsius and check whether it is boiling (100+ degrees), freezing (less than or equal to 0) or neither. Print out the response.

Include comments, handle exceptions, and test your code!

One possible solution

```
# This code asks a user the temperature of water in Celsius
# Then checks whether it is boiling (100+ degrees), freezing (less than or equal to 0) or neither
try:
    #convert the user's input into a float
    temp = float(input("What is the temperature of the water (in Celsius)? "))
except ValueError:
    #handle invalid input
    print("Sorry, that is not a valid number")
else:
    #print out the response
    print("The water is boiling" if temp >= 100
          else "The water is freezing" if temp <= 0
          else "The water is neither freezing nor boiling")
```


Let's chat

What is an example of a decision point or logic check in your job or personal life?

Next week

- Logic quiz due by next class
- Topics for next week
 - looping logic and how to apply it in Python